hw2

May 7, 2026

1 CS 229 - Homework 2: Confident Emoji Hunter

Submit **PDF** of completed notebook on Canvas.

Maximum points: 15

Enter your information below:

(full) Name: Vedant Deepak Borkute Student ID Number: 862552981

By submitting this notebook, I assert that the work below is my own work. Except where explicitly cited, none of the portions of this notebook are duplicated from anyone else's work.

2 Overview

Neural networks are often **overconfident** — they assign high probability to their predictions even when they're wrong. This is a serious problem in safety-critical applications (medical diagnosis, autonomous driving, etc.).

In this assignment you will: 1. Train an image classifier using frozen DINOv2 features 2. Measure how well calibrated its confidence is 3. Compare three methods for improving calibration: - **Temperature scaling** (post-hoc) - **MC Dropout** (inference-time) - **Label smoothing** (train-time)

These methods are evaluated **independently** — do not combine them.

2.0.1 The Emoji Hunter Task

Your dataset contains 128×128 natural images. Half have a Twemoji (e.g. fire, rocket, ghost, pizza) overlaid with varying size and transparency. Your job: classify which images contain an emoji. Some emojis are large and obvious; others are small and nearly transparent — making the model's confidence particularly interesting to study.

Complete all parts marked `TODO` and ensure all test cells produce the expected output.

2.1 Setup

Run these cells to load the model and data. **Do not modify.**

[91]:
```
%matplotlib inline
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt
import numpy as np
```

[92]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
device = torch.device('mps') if torch.backends.mps.is_available() else device
print(f'Using device: {device}')

torch.manual_seed(229)
```

```
Using device: cuda
```

[92]:
```
<torch._C.Generator at 0x7e99385bd310>
```

[93]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

2.2 Load Data

Download `hw2_data.pt` from Canvas and place it in the same directory as this notebook.

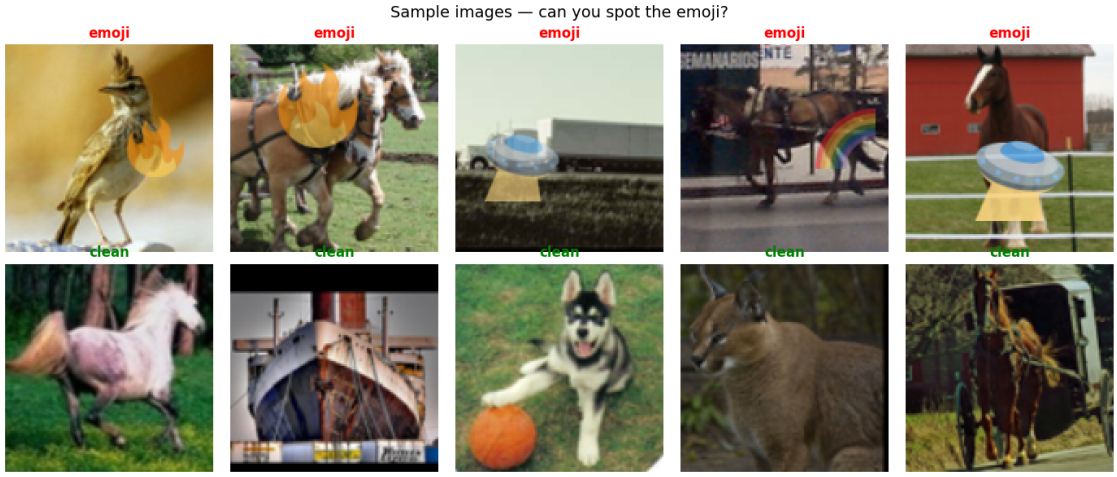
The dataset contains 128×128 natural images (from STL-10). Half have a Twemoji overlaid with varying size and transparency. Your task is to classify which images contain an emoji.

[94]:
```
data = torch.load('/content/drive/My Drive/CS229/hw2_data.pt',␣
 ↪weights_only=False)
train_images = data['train_images']   # (1200, 3, 128, 128) uint8
train_labels = data['train_labels']   # (1200,) long: 0=clean, 1=emoji
test_images = data['test_images']     # (400, 3, 128, 128) uint8
test_labels = data['test_labels']     # (400,) long

print(f'Train: {len(train_images)} images ({train_labels.sum().item()} emoji,␣
 ↪{(train_labels == 0).sum().item()} clean)')
print(f'Test:  {len(test_images)} images ({test_labels.sum().item()} emoji,␣
 ↪{(test_labels == 0).sum().item()} clean)')
print(f'Image shape: {train_images.shape[1:]}, dtype: {train_images.dtype}')
```

```
Train: 1200 images (598 emoji, 602 clean)
Test:  400 images (202 emoji, 198 clean)
Image shape: torch.Size([3, 128, 128]), dtype: torch.uint8
```

[95]:
```
# Visualize some examples
fig, axes = plt.subplots(2, 5, figsize=(14, 6))
fig.suptitle('Sample images - can you spot the emoji?', fontsize=14)
```

```
emoji_idxs = (train_labels == 1).nonzero().squeeze()[:5]
clean_idxs = (train_labels == 0).nonzero().squeeze()[:5]

for i, idx in enumerate(emoji_idxs):
    axes[0, i].imshow(train_images[idx].permute(1, 2, 0).numpy())
    axes[0, i].set_title('emoji', color='red', fontweight='bold')
    axes[0, i].axis('off')

for i, idx in enumerate(clean_idxs):
    axes[1, i].imshow(train_images[idx].permute(1, 2, 0).numpy())
    axes[1, i].set_title('clean', color='green', fontweight='bold')
    axes[1, i].axis('off')

plt.tight_layout()
plt.show()
```



2.3 Extract DINOv2 Features

We use a frozen DINOv2 backbone (ViT-Small/14) to extract 384-dimensional feature vectors from each image. These features capture rich visual representations without any task-specific training.

You will train only a small classifier head on top of these frozen features. **Do not modify this cell.**

[96]:
```
# Load DINOv2 ViT-Small/14 (downloads ~80MB on first run)
dino = torch.hub.load('facebookresearch/dinov2', 'dinov2_vits14')
dino = dino.to(device)
dino.eval()
print('DINOv2 ViT-S/14 loaded')

# ImageNet normalization constants
```

```
IMG_MEAN = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1).to(device)
IMG_STD = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1).to(device)


def extract_features(images, batch_size=64):
    """Extract DINOv2 CLS token features from uint8 image tensors."""
    features = []
    with torch.no_grad():
        for i in range(0, len(images), batch_size):
            batch = images[i:i + batch_size].float() / 255.0
            batch = F.interpolate(batch, size=224, mode='bilinear',␣
 ↪align_corners=False)
            batch = (batch.to(device) - IMG_MEAN) / IMG_STD
            feat = dino(batch)
            features.append(feat.cpu())
    return torch.cat(features)


print('Extracting train features...')
train_features = extract_features(train_images)
print('Extracting test features...')
test_features = extract_features(test_images)
print(f'Feature shape: {train_features.shape}')  # (1200, 384)
```

```
Using cache found in /root/.cache/torch/hub/facebookresearch_dinov2_main
```

```
DINOv2 ViT-S/14 loaded
Extracting train features…
Extracting test features…
Feature shape: torch.Size([1200, 384])
```

2.4 Provided Helper: Calibration Plot

This function plots a reliability diagram and prediction distribution. You will call it after computing calibration metrics. **Do not modify.**

[97]:
```
def plot_calibration(bin_confidences, bin_accuracies, bin_counts, ece,␣
 ↪title='Calibration'):
    """Plot reliability diagram and prediction distribution.

    Args:
        bin_confidences: (n_bins,) average confidence per bin
        bin_accuracies: (n_bins,) average accuracy per bin
        bin_counts: (n_bins,) number of predictions per bin
        ece: scalar expected calibration error
        title: plot title prefix
    """
    # Move to CPU/numpy in case tensors are on GPU
```

```
    bin_confidences = bin_confidences.detach().cpu()
    bin_accuracies = bin_accuracies.detach().cpu()
    bin_counts = bin_counts.detach().cpu()

    n_bins = len(bin_confidences)
    width = 1.0 / n_bins
    # Only plot bins that have data
    mask = bin_counts > 0

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))

    # Reliability diagram
    ax1.bar(bin_confidences[mask], bin_accuracies[mask], width=width * 0.8,
            alpha=0.7, color='steelblue', edgecolor='black', label='Model')
    ax1.plot([0, 1], [0, 1], 'r--', linewidth=1.5, label='Perfect calibration')
    ax1.set_xlabel('Confidence P(emoji)')
    ax1.set_ylabel('Fraction actually emoji')
    ax1.set_title(f'{title}\nECE = {ece:.4f}')
    ax1.legend()
    ax1.set_xlim(0, 1)
    ax1.set_ylim(0, 1)

    # Prediction distribution
    ax2.bar(bin_confidences[mask], bin_counts[mask], width=width * 0.8,
            alpha=0.7, color='coral', edgecolor='black')
    ax2.set_xlabel('Confidence P(emoji)')
    ax2.set_ylabel('Count')
    ax2.set_title('Prediction Distribution')

    plt.tight_layout()
    plt.show()
```

2.5 Part 1: Implement Calibration Metrics [3 points]

A well-calibrated classifier's confidence should match its actual accuracy. If a model says there's an 80% chance an image contains an emoji, then among all images it assigns 80% probability, about 80% should actually contain an emoji.

The **Expected Calibration Error (ECE)** measures how far a model is from perfect calibration. It works by: 1. Binning predictions by confidence level (e.g., 10 bins: 0–0.1, 0.1–0.2, ..., 0.9–1.0) 2. Computing the average accuracy and average confidence within each bin 3. Taking the weighted average of |accuracy − confidence| across bins, weighted by the fraction of samples in each bin

$$\text{ECE} = \sum_{b=1}^{B} \frac{n_b}{N} \left| \text{acc}(b) - \text{conf}(b) \right|$$

where n_b is the number of samples in bin b and N is the total number of samples. Empty bins ($n_b = 0$) contribute nothing to the sum — leave their confidence and accuracy as zero.

Important: binary calibration convention. For binary classification, we follow Guo et al., 2017 and use the **probability of the positive class** (emoji) as the confidence, not the max softmax probability. This allows confidence to range over the full [0, 1] interval — a prediction of $p(\text{emoji}) = 0.1$ means the model is 90% confident the image is clean. Using max softmax would collapse the range to [0.5, 1], hiding important calibration information.

Correspondingly, "accuracy" in each bin is the **fraction of images that actually contain an emoji**, and `labels` is simply the true label (1 = emoji, 0 = clean).

(For multi-class classification, confidence is typically taken as the max softmax probability and accuracy is the fraction of correct predictions. The binary convention above is specific to two-class problems.)

Implement `compute_calibration` below.

[98]:
```
def compute_calibration(confidences, labels, n_bins=10):
    """Compute calibration metrics: per-bin accuracy, confidence, counts, and␣
 ↪ECE.

    Args:
        confidences: (N,) tensor - predicted probability of the positive class␣
 ↪(emoji)
        labels: (N,) tensor - true labels (1 = emoji, 0 = clean)
        n_bins: number of confidence bins
    Returns:
        bin_confidences: (n_bins,) average confidence per bin
        bin_accuracies: (n_bins,) average accuracy (fraction positive) per bin
        bin_counts: (n_bins,) number of predictions per bin
        ece: scalar expected calibration error
    """
    labels = labels.to(confidences.device)
    # TODO [2 points]: Implement calibration binning and ECE computation
    # Hint: use torch.linspace(0, 1, n_bins + 1) for bin boundaries
    # For each bin, find predictions with confidence in that range,
    # compute average confidence and fraction of positives, then compute ECE.
    # Empty bins (no predictions in that range) should be left as zeros.
    bins = torch.linspace(0, 1, n_bins + 1).to(confidences.device)
    bin_confidences = torch.zeros(n_bins, device=confidences.device)
    bin_accuracies = torch.zeros(n_bins, device=confidences.device)
    bin_counts = torch.zeros(n_bins, device=confidences.device)

    for i in range(n_bins):
        if i == n_bins - 1:
            bin_mask = (confidences >= bins[i]) & (confidences <= bins[i+1])
        else:
            bin_mask = (confidences >= bins[i]) & (confidences < bins[i+1])
```

```
        bin_counts[i] = bin_mask.sum()
        if bin_counts[i] > 0:
            bin_confidences[i] = confidences[bin_mask].mean()
            bin_accuracies[i] = labels[bin_mask].float().mean()

    ece = (bin_counts * torch.abs(bin_accuracies - bin_confidences)).sum() /␣
↪labels.size(0)

    return bin_confidences, bin_accuracies, bin_counts, ece
```

2.5.1 Test your calibration function [1 point]

Verify your implementation on synthetic examples with known answers. All tests should pass.

[99]:
```
# Test 1: Well-calibrated - p(emoji)=0.75, 3/4 are actually emoji
conf1 = torch.tensor([0.75, 0.75, 0.75, 0.75])
lab1 = torch.tensor([1, 1, 1, 0])
_, _, _, ece1 = compute_calibration(conf1, lab1, n_bins=10)
print(f'Test 1 (well-calibrated): ECE = {ece1:.4f} (expected: 0.0000)')
assert abs(ece1) < 1e-4, f'Expected ~0.0, got {ece1}'

# Test 2: Overconfident - p(emoji)=0.95, but only 2/4 are emoji
conf2 = torch.tensor([0.95, 0.95, 0.95, 0.95])
lab2 = torch.tensor([1, 1, 0, 0])
_, _, _, ece2 = compute_calibration(conf2, lab2, n_bins=10)
print(f'Test 2 (overconfident):   ECE = {ece2:.4f} (expected: 0.4500)')
assert abs(ece2 - 0.45) < 1e-4, f'Expected ~0.45, got {ece2}'

# Test 3: Underconfident - p(emoji)=0.25 (model thinks "probably clean"), but␣
↪all are emoji
# Note: this test uses confidence < 0.5 - only possible because we use␣
↪p(positive),
# not max softmax (which would be clamped to [0.5, 1])
conf3 = torch.tensor([0.25, 0.25, 0.25, 0.25])
lab3 = torch.tensor([1, 1, 1, 1])
_, _, _, ece3 = compute_calibration(conf3, lab3, n_bins=10)
print(f'Test 3 (underconfident):  ECE = {ece3:.4f} (expected: 0.7500)')
assert abs(ece3 - 0.75) < 1e-4, f'Expected ~0.75, got {ece3}'

print('\nAll calibration tests passed!')
```

```
Test 1 (well-calibrated): ECE = 0.0000 (expected: 0.0000)
Test 2 (overconfident):   ECE = 0.4500 (expected: 0.4500)
Test 3 (underconfident):  ECE = 0.7500 (expected: 0.7500)

All calibration tests passed!
```

2.6 Part 2: Train Base Classifier [3 points]

Define a small MLP classifier head that takes DINOv2 features (384-dim) and outputs binary logits (emoji vs. clean).

Architecture [1 point]: 1. `nn.Linear(384, 128)` → `nn.ReLU()` 2. `nn.Dropout(0.5)` 3. `nn.Linear(128, 2)` (output logits)

The Dropout layer is important — we'll use it later for MC Dropout.

Training [2 points]: Use `Adam` optimizer with `lr=1e-3`, `CrossEntropyLoss`, batch size 64, 20 epochs. Print loss and test accuracy every 5 epochs.

[100]:
```
class EmojiClassifier(nn.Module):
    # TODO [1 point]: Define the MLP classifier head as described above
    def __init__(self, input_dim=384, hidden_dim=128, n_classes=2, dropout=0.5):
        super().__init__()
        self.layers = nn.Sequential(nn.Linear(384, 128), nn.ReLU(), nn.
↪Dropout(0.5), nn.Linear(128,2))
        # TODO: define layers

    def forward(self, x):
        # TODO: implement forward pass, return raw logits
        return self.layers(x)
        pass
```

2.6.1 Test your model

The cell below should print output shape `torch.Size([4, 2])` with dtype `torch.float32`.

[101]:
```
model = EmojiClassifier().to(device)
print(f'Model:\n{model}\n')
test_out = model(torch.randn(4, 384).to(device))
print(f'Output shape: {test_out.shape}, dtype: {test_out.dtype}')
```

```
Model:
EmojiClassifier(
  (layers): Sequential(
    (0): Linear(in_features=384, out_features=128, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.5, inplace=False)
    (3): Linear(in_features=128, out_features=2, bias=True)
  )
)

Output shape: torch.Size([4, 2]), dtype: torch.float32
```

[102]:
```
# TODO [2 points]: Training loop
# Use Adam optimizer with lr=1e-3, CrossEntropyLoss, batch_size=64, 20 epochs
# Print loss and test accuracy every 5 epochs
```

```
# Hint: create a TensorDataset and DataLoader from train_features and␣
 ↪train_labels

n_epochs = 20
batch_size = 64
learning_rate = 1e-3

# TODO: create DataLoader, optimizer, loss function
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
criterion = torch.nn.CrossEntropyLoss()
train_dataset = TensorDataset(train_features, train_labels)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

# TODO: training loop
for epoch in range(1, n_epochs + 1):
    model.train()
    total_loss = 0.0
    for batch_features, batch_labels in train_loader:
        batch_features, batch_labels = batch_features.to(device), batch_labels.
 ↪to(device)
        optimizer.zero_grad()
        outputs = model(batch_features)
        loss = criterion(outputs, batch_labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * batch_features.size(0)

    avg_loss = total_loss / len(train_dataset)

    if epoch % 5 == 0:
        model.eval()
        with torch.no_grad():
            test_outputs = model(test_features.to(device))
            test_preds = test_outputs.argmax(dim=1)
            test_labels_dev = test_labels.to(test_preds.device)
            test_acc = (test_preds == test_labels_dev).float().mean().item()
        print(f'Epoch {epoch}/{n_epochs} - Loss: {avg_loss:.4f} - Test Accuracy:
 ↪ {test_acc:.4f}')
```

```
Epoch 5/20 - Loss: 0.2900 - Test Accuracy: 0.7725
Epoch 10/20 - Loss: 0.1247 - Test Accuracy: 0.7775
Epoch 15/20 - Loss: 0.0553 - Test Accuracy: 0.7825
Epoch 20/20 - Loss: 0.0333 - Test Accuracy: 0.7800
```

2.7 Part 3: Calibration Analysis of Base Model [2 points]

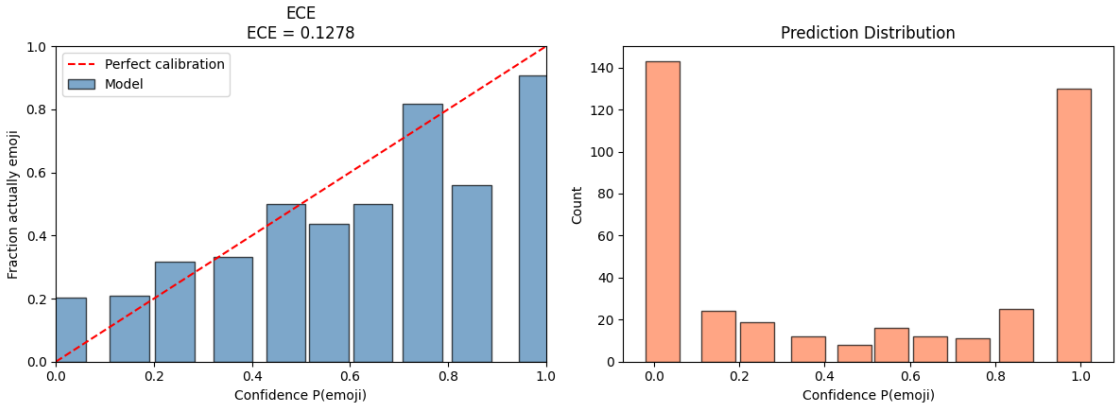
Now evaluate how well calibrated your base model is.

3a [1 point]: Get the model's predictions and confidences on the test set, then compute and plot the calibration curve using your `compute_calibration` function and the provided `plot_calibration` helper. Print the ECE.

Remember: use `model.eval()` and `torch.no_grad()` for evaluation. Confidence = `probs[:, 1]` (probability of the positive class, as described in Part 1).

3b [1 point]: Visualize 3 images where the model is **most confident** they contain an emoji and 3 where it is **least confident** (among those it predicts as emoji). Do the confidence levels make intuitive sense?

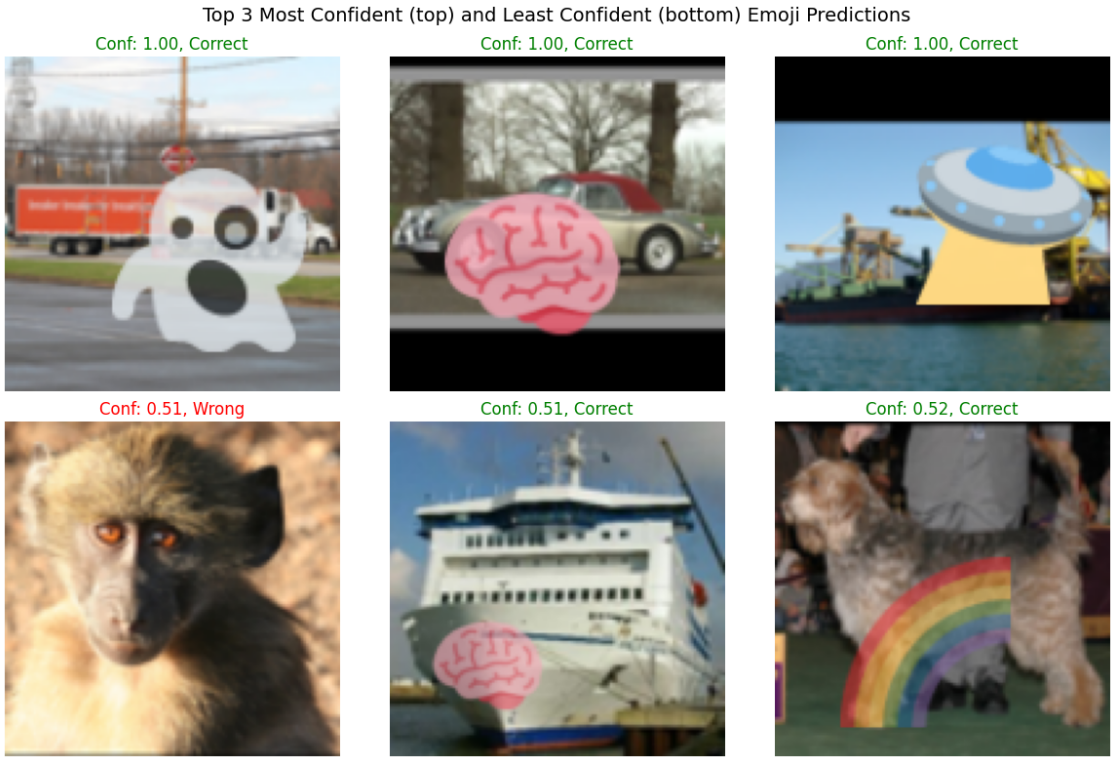
[103]:
```
# TODO 3a [1 point]: Get predictions, confidences, and plot calibration
# 1. Set model to eval mode
# 2. Get logits on test_features (with no_grad)
# 3. Compute softmax probabilities
# 4. Confidence = probs[:, 1] - the probability of emoji (positive class)
# 5. Call compute_calibration(confidences, test_labels) and plot_calibration
# Also compute accuracy for reporting: (probs.argmax(dim=1) == test_labels).
 ↪float().mean()

# Save test_logits - you'll need them for temperature scaling later!
# test_logits = ...

model.eval()
with torch.no_grad():
    test_logits = model(test_features.to(device))
    test_probs = F.softmax(test_logits, dim=1)
    confidences = test_probs[:, 1]  # Probability of emoji class
    test_labels_dev = test_labels.to(device)
    bin_confidences, bin_accuracies, bin_counts, ece =␣
 ↪compute_calibration(confidences, test_labels_dev)
    plot_calibration(bin_confidences, bin_accuracies, bin_counts, ece,␣
 ↪title='ECE')
    test_acc = (test_probs.argmax(dim=1) == test_labels_dev).float().mean().
 ↪item()
    print(f'Test Accuracy: {test_acc:.4f}, ECE: {ece:.4f}')
```




```
Test Accuracy: 0.7800, ECE: 0.1278
```

[104]:
```python
# TODO 3b [1 point]: Visualize most and least confident emoji predictions
# Among test images the model predicts as emoji (class 1),
# show the 3 most confident and 3 least confident.
# Include the confidence value and whether the prediction was correct in the title.

# Hint: filter to predictions where test_preds == 1,
# sort by confidence, show top 3 and bottom 3 using test_images
test_preds_cpu = test_probs.argmax(dim=1).detach().cpu()
confidences_cpu = confidences.detach().cpu()
test_labels_cpu = test_labels.detach().cpu()

emoji_mask = test_preds_cpu == 1
emoji_confidences = confidences_cpu[emoji_mask]
emoji_labels = test_labels_cpu[emoji_mask]
emoji_images = test_images[emoji_mask]
sorted_indices = torch.argsort(emoji_confidences)
top3_indices = sorted_indices[-3:]
bottom3_indices = sorted_indices[:3]

fig, axes = plt.subplots(2, 3, figsize=(12, 8))
for i, idx in enumerate(top3_indices):
    conf = emoji_confidences[idx].item()
    correct = (emoji_labels[idx] == 1).item()
    title = f'Conf: {conf:.2f}, {"Correct" if correct else "Wrong"}'
    axes[0, i].imshow(emoji_images[idx].permute(1, 2, 0).numpy())
    axes[0, i].set_title(title, color='green' if correct else 'red')
    axes[0, i].axis('off')
for i, idx in enumerate(bottom3_indices):
    conf = emoji_confidences[idx].item()
    correct = (emoji_labels[idx] == 1).item()
```

```
    title = f'Conf: {conf:.2f}, {"Correct" if correct else "Wrong"}'
    axes[1, i].imshow(emoji_images[idx].permute(1, 2, 0).numpy())
    axes[1, i].set_title(title, color='green' if correct else 'red')
    axes[1, i].axis('off')
plt.suptitle('Top 3 Most Confident (top) and Least Confident (bottom) Emoji␣
 ↪Predictions', fontsize=14)
plt.tight_layout()
plt.show()
```



The confidence levels do make intuitive sense as the wrong predictions are generally harder for a human to even classify.

2.8 Part 4: Temperature Scaling [3 points]

Temperature scaling divides logits by a scalar $T > 0$ before applying softmax:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- $T = 1$: no change (original model)
- $T > 1$: softens probabilities (reduces overconfidence)
- $T < 1$: sharpens probabilities (increases confidence)

Temperature scaling is a **post-hoc** method — it does not require retraining. You apply it to the logits from your already-trained base model.

4a [1 point]: Write a function that applies temperature scaling to logits and returns probabilities.

4b [1 point]: Try different values of T (e.g., 0.5, 1.0, 1.5, 2.0, 3.0, 5.0) and find one that improves ECE. Print the ECE for each T you try.

4c [1 point]: Plot the calibration curve for your best T.

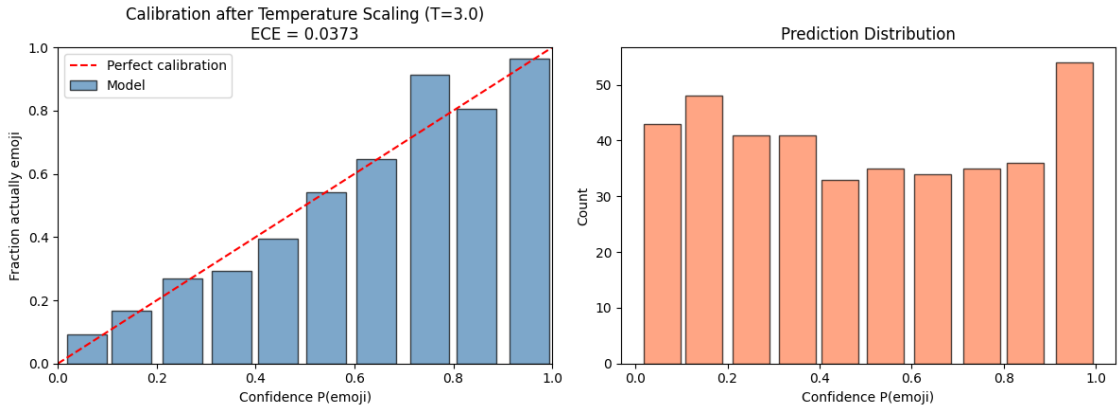
[105]:
```
# TODO 4a [1 point]: Implement temperature scaling
def apply_temperature(logits, T):
    """Apply temperature scaling to logits.

    Args:
        logits: (N, C) raw model logits
        T: temperature scalar (> 0)
    Returns:
        (N, C) scaled probabilities
    """
    # TODO: divide logits by T, then apply softmax
    scaled_logits = logits / T
    scaled_probs = F.softmax(scaled_logits, dim=1)
    return scaled_probs

    pass
```

[106]:
```
# TODO 4b [1 point]: Try different T values on test_logits, print ECE for each
# Find the T that gives the lowest ECE
T_values = [0.5, 1.0, 1.5, 2.0, 3.0, 5.0]
best_T = None
best_ece_t = float('inf')
for T in T_values:
    scaled_probs = apply_temperature(test_logits, T)
    confidences = scaled_probs[:, 1]  # Probability of emoji class
    test_labels_dev = test_labels.to(confidences.device)
    bin_confidences, bin_accuracies, bin_counts, ece_t =␣
 ↪compute_calibration(confidences, test_labels_dev)
    print(f'Temperature: {T:.1f}, ECE: {ece_t:.4f}')
    if ece_t < best_ece_t:
        best_ece_t = ece_t
        best_T = T

        best_bin_confidences = bin_confidences
        best_bin_accuracies = bin_accuracies
        best_bin_counts = bin_counts
print(f'Best temperature: {best_T:.1f} with ECE: {best_ece_t:.4f}')
```

```
Temperature: 0.5, ECE: 0.1696
```

```
Temperature: 1.0, ECE: 0.1278
Temperature: 1.5, ECE: 0.0919
Temperature: 2.0, ECE: 0.0687
Temperature: 3.0, ECE: 0.0373
Temperature: 5.0, ECE: 0.0906
Best temperature: 3.0 with ECE: 0.0373
```

[107]:
```
# TODO 4c [1 point]: Plot calibration curve with your best T
# Use plot_calibration with an appropriate title
plot_calibration(best_bin_confidences,best_bin_accuracies,best_bin_counts,best_ece_t,␣
  ↪title=f'Calibration after Temperature Scaling (T={best_T:.1f})'
)
```



2.9 Part 5: MC Dropout [2 points]

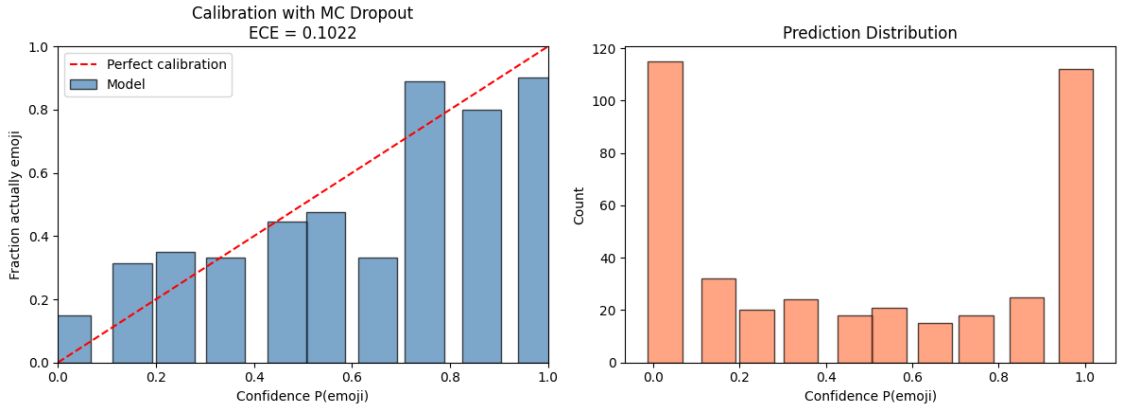
Monte Carlo Dropout estimates prediction uncertainty by running multiple forward passes with dropout **enabled** at test time, then averaging the softmax outputs.

Implementation: 1. Put the model in eval mode: `model.eval()` 2. Re-enable **only** the dropout layers: use `model.apply(...)` to set `nn.Dropout` modules back to train mode 3. Run 30 forward passes on the test features 4. Average the **softmax probabilities** (not logits!) across all passes 5. Compute and plot calibration on the averaged probabilities

Why eval + dropout train? If your model had batch normalization, you'd want those layers to stay in eval mode (using stored statistics). Here we only have dropout, so `model.train()` would also work — but the pattern below is best practice.

```
model.eval()
def enable_dropout(m):
    if isinstance(m, nn.Dropout):
        m.train()
model.apply(enable_dropout)
```

[108]:
```
# TODO [2 points]: Implement MC Dropout
# 1. Set model to eval, then re-enable only dropout (code pattern shown above)
# 2. Run 30 forward passes, collecting softmax probabilities from each
# 3. Average the probab
# ilities across passes
# 4. Compute accuracy, calibration, and plot
model.eval()
model.apply(lambda m: m.train() if isinstance(m, nn.Dropout) else None)


N_PASSES = 30

for i in range(N_PASSES):
    with torch.no_grad():
        test_logits = model(test_features.to(device))
        test_probs = F.softmax(test_logits, dim=1)
        if i == 0:
            all_probs = test_probs.unsqueeze(0)  # (1, N, C)
        else:
            all_probs = torch.cat((all_probs, test_probs.unsqueeze(0)), dim=0)
 ↪# (i+1, N, C)

avg_probs = all_probs.mean(dim=0)  # (N, C)
confidences = avg_probs[:, 1]  # Probability of emoji class
test_labels_dev = test_labels.to(avg_probs.device)
bin_confidences, bin_accuracies, bin_counts, ece_mc =
 ↪compute_calibration(confidences, test_labels_dev)
plot_calibration(bin_confidences, bin_accuracies, bin_counts, ece_mc,
 ↪title='Calibration with MC Dropout')
test_acc = (avg_probs.argmax(dim=1) == test_labels_dev).float().mean().item()
print(f'MC Dropout Test Accuracy: {test_acc:.4f}, ECE: {ece_mc:.4f}')
```




```
MC Dropout Test Accuracy: 0.7775, ECE: 0.1022
```

2.10 Part 6: Label Smoothing [2 points]

Label smoothing is a **train-time** regularization technique that replaces hard labels (0 or 1) with soft labels. Instead of training on $y = [0, 1]$, you train on $y = [\alpha/K, 1 - \alpha + \alpha/K]$ where α is the smoothing factor and K is the number of classes.

This discourages the model from being overly confident in its predictions.

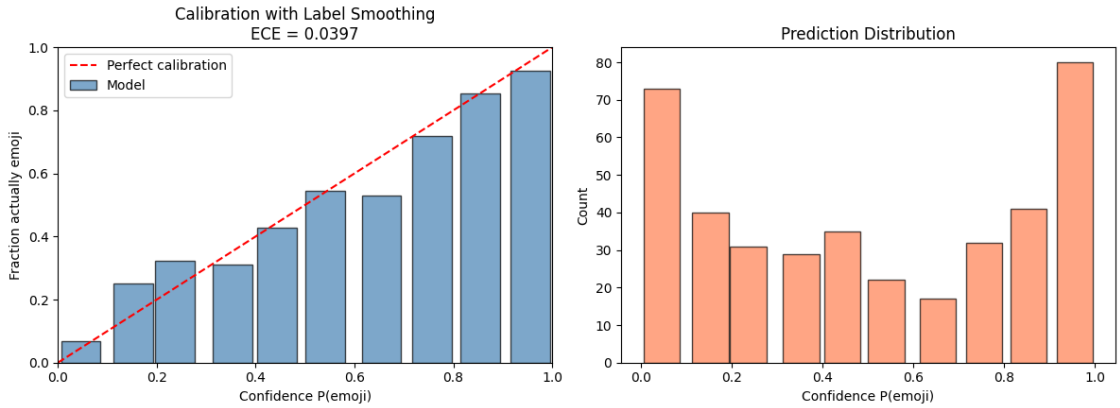
PyTorch makes this easy: `nn.CrossEntropyLoss(label_smoothing=0.1)`

Retrain a fresh model from scratch with label smoothing and evaluate its calibration. This is **independent** of the temperature scaling and MC Dropout results — do not combine methods.

[109]:
```
# TODO [2 points]: Retrain with label smoothing and evaluate calibration
# 1. Create a new EmojiClassifier (fresh weights)
# 2. Train with CrossEntropyLoss(label_smoothing=0.1) - same hyperparameters as␣
 ↪before
# 3. Evaluate: get test predictions, compute calibration, plot
model_ls = EmojiClassifier().to(device)
optimizer_ls = torch.optim.Adam(model_ls.parameters(), lr=learning_rate)
criterion_ls = torch.nn.CrossEntropyLoss(label_smoothing=0.1)
train_dataset_ls = TensorDataset(train_features, train_labels)
train_loader_ls = DataLoader(train_dataset_ls, batch_size=batch_size,␣
 ↪shuffle=True)
n_epochs_ls = 20
for epoch in range(1, n_epochs_ls + 1):
    model_ls.train()
    total_loss = 0.0
    for batch_features, batch_labels in train_loader_ls:
        batch_features, batch_labels = batch_features.to(device), batch_labels.
 ↪to(device)
        optimizer_ls.zero_grad()
        outputs = model_ls(batch_features)
        loss = criterion_ls(outputs, batch_labels)
        loss.backward()
        optimizer_ls.step()
        total_loss += loss.item() * batch_features.size(0)

    avg_loss = total_loss / len(train_dataset_ls)

    if epoch % 5 == 0:
        model_ls.eval()
        with torch.no_grad():
            test_outputs = model_ls(test_features.to(device))
            test_preds = test_outputs.argmax(dim=1)
            test_labels_dev = test_labels.to(test_preds.device)
            test_acc = (test_preds == test_labels_dev).float().mean().item()
```

```
        print(f'[Label Smoothing] Epoch {epoch}/{n_epochs_ls} - Loss: {avg_loss:
 ↪.4f} - Test Accuracy: {test_acc:.4f}')

model_ls.eval()
with torch.no_grad():
    test_logits_ls = model_ls(test_features.to(device))
    test_probs_ls = F.softmax(test_logits_ls, dim=1)
    confidences_ls = test_probs_ls[:, 1]  # Probability of emoji class
    test_labels_dev = test_labels.to(device)
    bin_confidences_ls, bin_accuracies_ls, bin_counts_ls, ece_ls =␣
 ↪compute_calibration(confidences_ls, test_labels_dev)
    plot_calibration(bin_confidences_ls, bin_accuracies_ls, bin_counts_ls,␣
 ↪ece_ls, title='Calibration with Label Smoothing')
    test_acc_ls = (test_probs_ls.argmax(dim=1) == test_labels_dev).float().
 ↪mean().item()
    print(f'Label Smoothing Test Accuracy: {test_acc_ls:.4f}, ECE: {ece_ls:.
 ↪4f}')
```

```
[Label Smoothing] Epoch 5/20 - Loss: 0.3926 - Test Accuracy: 0.7650
[Label Smoothing] Epoch 10/20 - Loss: 0.2853 - Test Accuracy: 0.7975
[Label Smoothing] Epoch 15/20 - Loss: 0.2588 - Test Accuracy: 0.7875
[Label Smoothing] Epoch 20/20 - Loss: 0.2423 - Test Accuracy: 0.7800
```

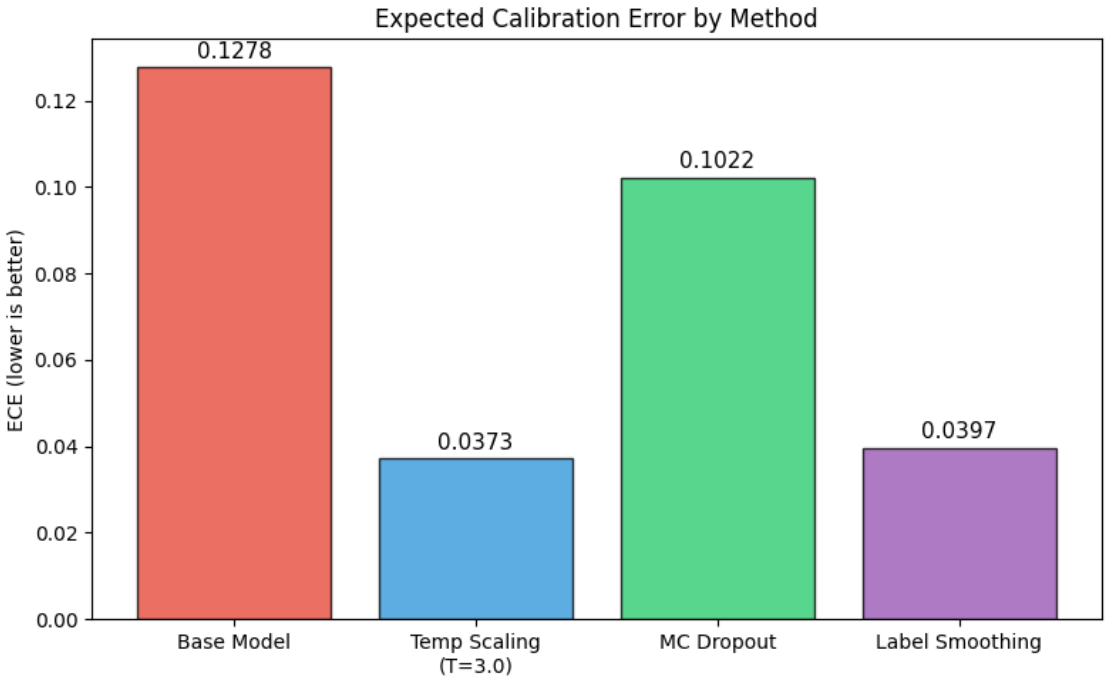



```
Label Smoothing Test Accuracy: 0.7800, ECE: 0.0397
```

2.11 Summary Comparison

Run this cell to see a side-by-side comparison of all methods. Make sure the variables from each part are still in scope.

[110]:
```
# Summary comparison - fill in your variable names if different
# This cell assumes you named your ECE values: ece, ece_t, ece_mc, ece_ls
# and your best temperature: best_T
# Adjust variable names if needed.

methods = ['Base Model', f'Temp Scaling\n(T={best_T})', 'MC Dropout', 'Label␣
 ↪Smoothing']
eces = [ece.cpu(), best_ece_t.cpu(), ece_mc.cpu(), ece_ls.cpu()]

fig, ax = plt.subplots(figsize=(8, 5))
colors = ['#e74c3c', '#3498db', '#2ecc71', '#9b59b6']
bars = ax.bar(methods, eces, color=colors, edgecolor='black', alpha=0.8)
ax.set_ylabel('ECE (lower is better)')
ax.set_title('Expected Calibration Error by Method')
for bar, e in zip(bars, eces):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.002,
            f'{e:.4f}', ha='center', fontsize=11)
plt.tight_layout()
plt.show()
```



2.12 Extra Credit

Extra credit is submitted separately on Canvas using the **HW 2 (EC)** assignment. You must submit a PDF (made with LaTeX) outlining the problem, your approach, results (including tables or figures), and a discussion of results. Also submit your code (.py or .ipynb). Extra credit is graded on quality, generally 1–5 points, though could be higher for novel research directions. If you have a direction, feel free to pitch it!

Compare at least **three** additional calibration improvement methods beyond those in the homework. Possible directions include (but are not limited to):

- **Mixup** (Zhang et al., 2018) — train on convex combinations of input pairs
- **Platt Scaling** — learn a logistic regression on top of logits using a held-out validation set
- **L2 Regularization / Weight Decay** — does stronger regularization improve calibration?
- **Data Augmentation** — color jitter, random crops, etc. on the raw images before feature extraction
- **Focal Loss** (Lin et al., 2017) — downweight easy examples during training
- **Ensemble Methods** — train multiple models with different seeds, average their predictions
- **Deep Ensembles** (Lakshminarayanan et al., 2017) — do ensembles improve calibration more than MC Dropout?

2.13 Submission

Export this notebook to PDF and submit on Canvas.

```
!jupyter nbconvert --to pdf --output=yourname_hw2.pdf hw2.ipynb
```